

Algorithm Design HW2

Due Date 29 October 23:55*

- 1) Solve the recurrence relations below and express them using θ notation
 - a) $T(n)=9T(n-1)-20T(n-2)$, $T(0)=0$, $T(1)=1$
 - b) $T(n)=2T(n/2) + n\log n$, $T(1)=1$
 - c) $T(n)=T(n-1)+n$, $T(1)=1$
- 2) Express the code snippet below as a recurrence relation and find the complexity at worse case in terms of O notation.

```
f(array A[1 . . i] , array B[1 . . j]):  
    if (i<1 or j<1) :  
        return 0  
    elif A[i]==B[j]:  
        return 1+ f(A[1 . . (i-1)] , B[1 . . (j-1)])  
    else:  
        return max(f(A[1 . . i] , B[1 . . (j-1)]), f(A[1 . . (i-1)] , B[1 . . j]))
```

- 3) Let A be an array of sorted numbers. Let B be the k times shifted version of A . Eg . $A = \{1,4,7,12,34,67\}$ and $B = \{34,67,1,4,7,12\}$. In this example B is 2 times shifted. Design an algorithm that finds the largest element of B in $O(\log n)$ time. This is a programming question implement the related function in the python file that is shared.

- 4) Let A be an array of integers whose elements are 1 or 2 or 3 only. Find an algorithm that sorts this array in $O(n)$ time. This is a programming

*Upload pictures of 1. and 2. questions and the python file to moodle due 29 October 23:55 and bring the hardcopy of 1. and 2. questions to room 108 just after the holiday. Solutions implemented in an other programming language or pseudocode solutions to 3,4,5 will not be evaluated. So only working python functions will be evaluated for the second part.

question implement the related function in the python file that is shared.

5) Let A be an array of n sorted distinct integers between 1 and m (1 and m are inclusive) where $n < m$. Find the smallest integer k such that $k < m$ and k does not exist in A . Your algorithm should work in $O(\log n)$ time. This is a programming question implement the related function in the python file that is shared.